

How we read six phrases of the SOM

Milestone 3 evidence, cross-cutting. Six phrases of the Statement of Milestones describe something that this milestone implements **with a different shape than the words suggest**, or not at all. This page says so directly, phrase by phrase, with the code and the test behind each reading. **The readings are ours.** We wrote each one down as a decision before this package was assembled (the decision log, `DECISIONS.md`, is in Spanish; the IDs are cited so they can be checked).

Why the shape differs

The SOM was written before the design deliverables of Milestones 1 and 2. Those deliverables (lifecycle diagram, information architecture and permission matrix, component library, the 53-surface implementation backlog, the screenshot catalog, the backend and smart-contract design plan) are the SOM's intent worked out in detail and already accepted by Catalyst. When a phrase of the SOM asks for something that none of them shows, we built what the deliverables show and recorded why (D-090), instead of adding a field, a screen or a contract branch that nothing in the accepted design asked for.

There is a second reason that applies to the smart contract specifically: the validator is already deployed on Preprod and its script hash backs the 180 transaction IDs, the walkthrough video and the validator coverage report in this package. Changing it now would invalidate that evidence.

1 · "Plutus V2 state machine"

What exists	The validator is written in Aiken and compiled to Plutus V3 (<code>contracts/plutus.json</code> , <code>"plutusVersion": "v3"</code>), the current Plutus version since the Chang hard fork. Nothing in the design needs V2
Where	<code>contracts/validators/stage.ak</code> , <code>contracts/lib/propnexus/fsm.ak</code>

2 · "Parameterized roles" and "configurable signers"

The SOM	<i>"Smart-contract suite (Plutus V2 state machine) with parameterized roles"; "supports ≥8 stages with configurable signers/percentages"</i>
What exists on-chain	The validator enforces sequence and time under one key. It takes a single parameter, <code>admin</code> (the platform's service key), and every transition requires that signature (<code>stage.ak</code> , <code>spend</code> and <code>mint</code>). Role enforcement lives one layer up, in the API, by design: see <i>Why</i>
What exists off-chain	The roles. Who may request each transition is fixed per transition and enforced by the API before anything is signed: the developer moves a stage to <code>InProgress</code> (first evidence upload, or "resume" after an observation); only the certifier can move it to <code>Completed</code> or <code>Observed</code> . A platform admin can make any legal transition. Any other request is rejected with 403 STAGE_TRANSITION_FORBIDDEN . The user who requested each transition is recorded in the append-only audit log, and the proof object of each anchored file returns the user who uploaded it as <code>signerUserId</code>

Why	The smart-contract design deliverable accepted in Milestone 2 (<i>Backend and smart contracts design and implementation plan</i> , §13.3) considers a phased approach appropriate: "beginning with compact anchoring transactions or simple commitment recording, evaluating where stronger on-chain enforcement offers meaningful value, and introducing richer validator-based logic only where justified by product, trust, or governance requirements". Its §13.1 adds that "the architecture does not require the full migration of business logic to on-chain execution". Role checks inside the validator are not justified yet, for a concrete reason: the whitepaper accepted in Milestone 1 (§System Overview) states that "all blockchain interactions are performed by operator-controlled backend services". With one operator signing, roles inside the validator would check nothing: every transaction carries the same key, so a role declared in it would be the operator's word, checked against nothing. What the chain proves is sequence and time : each transition spends the previous UTxO of the stage's thread, so the history cannot be rewritten afterwards, not even by the operator. What a professional attested lives off-chain, in the audit log. No design deliverable shows a screen or field to reassign who authorizes a transition per project, so it is not configurable: it is fixed and complete (D-020, D-058, D-092)
When it changes	Roles in the validator start to mean something when a role signs with its own key: the certifier co-signing the transaction from their own wallet (CIP-30), the model already chosen for professional signatures (D-009). That is the next phase of §13.3, not part of this milestone. Milestone 4 gives it a concrete trigger: its pilot metrics include "≥120 unique wallets across pilots", which cannot be reached while one key signs every transaction. How that target is met is an open decision for Milestone 4; any change to the validator changes its script hash, so it is taken together with the other script changes recorded for mainnet
Tests	<code>apps/api/test/stage-transitions.test.ts</code> ("403 al pedir Completed"), <code>apps/api/test/orpc-client-stages.test.ts</code> ("un developer que pide Completed recibe STAGE_TRANSITION_FORBIDDEN"), <code>apps/api/test/evidence-anchor.test.ts</code> (<code>signerUserId</code> in the proof object)
≥8 stages	Met: the default catalog has 10 stages (<code>DEFAULT_STAGE_CATALOG</code> , <code>packages/shared/src/stage.ts</code>), each with its own on-chain thread (one thread token per stage). The validator has no upper limit

3 · "Configurable percentages"

What exists	Progress is derived : completed stages over total stages of the project (<code>avanceDeStages()</code> , <code>apps/web/src/lib/stageProgress.ts</code>). It is not declared, not stored, and no stage weighs more than another
Why	The only design reference for project progress, screenshot 45 (developer progress), shows one number and one bar per project, with no per-stage weighting. An early version had a declared <code>progressPercentage</code> per stage; it was removed (migration <code>0003_drop_stage_progress.sql</code>) because no deliverable asked for it, and a declared percentage that the platform cannot check reads as a claim it cannot back (D-090, D-091)

4 · "Timeouts" and "fallback branches"

The SOM	"state machine ... with ... timeouts, and fallback branches"
What exists	Neither is in the validator, and the canonical lifecycle diagram of Milestone 1 has none: four states, transitions only by event. On Cardano nothing happens on-chain without someone submitting a signed transaction, so a timeout that "fires by itself" would need an off-chain actor anyway. We read both terms as properties of the anchoring pipeline , which is that off-chain actor (D-089):
"Fallback branch"	If anchoring fails (the anchoring service throws, or its configuration is broken), the off-chain record is written anyway and the anchor is stored as <code>Failed</code> instead of losing the upload. A stage whose on-chain thread could not be created can be re-anchored with <code>POST /projects/:id/stages/:stageId/retry-anchor</code> while it is still <code>Pending</code>

"Timeout"	An anchor that is not yet confirmed blocks nothing: it stays <code>Pending</code> and is promoted to <code>Confirmed</code> when it is next read (every screen that shows an anchor reconciles its own scope first) or by <code>POST /evidence/reconcile</code> , which a daily GitHub Actions job runs against production and which fails if it finds an inconsistent thread. There is no fixed deadline after which a <code>Pending</code> anchor changes state on its own; we say so plainly rather than call the daily job a timeout
Tests	<code>apps/api/test/evidence-upload.test.ts</code> ("si el anclaje falla (D-059), el lote igual entra: 201, archivos conservados y anchor Failed"), <code>apps/api/test/stage-transitions.test.ts</code> (<code>describe("POST /projects/:id/stages/:stageId/retry-anchor")</code>), including two concurrent retries minting at most once), <code>apps/api/test/reconcile.test.ts</code> , <code>apps/api/test/reconcile-on-read.test.ts</code>

5 · "Rejects unsigned evidence"

What the platform does not do	It does not verify cryptographic signatures on documents (PDF signatures, for instance). PropNexus does not certify or validate anything; it can only support four statements: <i>this file has this hash</i> , <i>it was recorded at this time</i> , <i>it declares that it comes from this external authority</i> , and <i>this person attested to having reviewed it</i> (D-026)
What "unsigned" means here	Evidence that declares itself authoritative (issued by an external authority, e.g. a building permit) but does not say which authority issued it (<code>issuingAuthority</code>) (D-028, D-084)
What is rejected, and where	(1) On upload: an authoritative file without an issuing authority is rejected with 400 EVIDENCE_UNATTRIBUTED , with no database row and no stored file left behind. (2) On edit: marking existing evidence authoritative without an authority, or removing the authority from authoritative evidence, gets the same 400 and changes nothing. (3) On completion: a stage cannot move to <code>Completed</code> while any of its evidence is authoritative and unattributed: 409 STAGE_EVIDENCE_UNATTRIBUTED (a safety net for rows recorded before rule 1 existed). The three checks share one rule, <code>evidenciaSinAtribuir</code> in <code>packages/shared/src/documents.ts</code> . Non-authoritative evidence (photos, site reports) can always be uploaded
Tests	<code>apps/api/test/evidence-upload.test.ts</code> , <code>describe("POST .../evidence · evidencia authoritative sin autoridad emisora (D-028)")</code> : "400 EVIDENCE_UNATTRIBUTED si se declara authoritative='on' sin issuingAuthority, sin fila ni huérfano", the same with a whitespace-only authority, the three <code>PATCH /evidence/:id</code> cases, and the accepted case with an authority. <code>apps/api/test/stage-transitions.test.ts</code> : "409 al completar con una evidencia autoritativa sin decir quién la emitió"
CI	Rules (1) and (2) were added on 2026-10-05, after the CI run that <code>1-repo-ci-tests/test-report.pdf</code> documents. The first green CI run that includes them is run 37377261309

6 · "Telemetry for ... error budgets"

The error-budget target, the indicator it is measured on, and where the data comes from are in [5-ops/monitoring-screenshots.pdf](#), section "How this also covers Output 4's telemetry".